
Oliver Krauss

Introduction

ALD | MTD.ba VZ SS26

Hagenberg 06.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Course Instructor

Oliver Krauss

- SE Bachelor & Master in Hagenberg
- Member of research group AIST since 2012
- Lectures since 2015
- oliver.krauss@fh-hagenberg.at
- +43-(0)50804/27195
- Office: FH Hagenberg - FH1.233



What we cover in Algorithmic Thinking

1. OOP Fundamentals & JavaFX Graphics (Classes, Inheritance, Interfaces)
2. Generics & Java Collections Framework
3. Data Structures (Stack, Queue, Trees, Maps)
4. State Machines & Game AI
5. DAO, 3-Layer Architecture & Networking
6. AI-Assisted Coding & Design Patterns

Organizational matters I

Attendance and participation

- Attendance in lectures is NOT mandatory
- Handing in the exercises IS mandatory
- Participation (if attending) is mandatory
- Both attendance and participation can only influence your grade positively

Organizational matters II

Teaching Method

- Slide Presentations
- Live coding demos & practical work with your participation
- Hands-on projects: shapes, games, and media applications
- All Material on Moodle

Organizational matters III

Grading

- Lecture: Written **exam** (60min. ALD VO grade)
- Exercises: **Homework** via Moodle + **Short Tests** (ALD UE grade)
 - **Homework:** 8 assignments (A01--A08), 24 points each
 - **Short Tests:** 3 tests offered, only best 2 count
 - Handing an assignment in late → -4 points / day
 - Deadlines will be extended only in case of emergency
 - The entire group may ask for a deadline extension
 - You can **skip ONE assignment**. Assignments only count as submitted if you reach at least 25% of points

Organizational matters IV

The Effort:

- This Class is worth **2.5 ECTS** (x2 → VO, UE)
- You should spend 62.5h on this class (In Austria 1 ECTS = 25h of work)
- 36 hours in class (if you attend!); 26.5 hours at home
- Please document your hours. It will allow me to align the class better next year.

Tools we use

Development

- **IntelliJ IDEA** (Ultimate) -- IDE
- **Maven** -- Build tool & dependencies
- **JavaFX** -- Graphics library
- **JUnit 5** -- Unit testing
- **GIT** -- Version control

AI & Data

- **GitHub Copilot** -- AI coding
- **MariaDB** -- Relational database
- **Java HttpClient** -- REST API access

Algorithmic Thinking -- Why it matters

Building on Modern Code

- In Semester 1 you learned **procedural** Java: variables, methods, arrays, loops
- Now we move to **object-oriented** thinking: classes, inheritance, polymorphism
- Same language (Java), fundamentally different way of structuring code

What you will be able to do

- Design and implement class hierarchies for real-world problems
- Build graphical applications with JavaFX
- Use data structures (lists, trees, maps) to solve complex problems
- Connect to databases and REST APIs
- Program game AI with state machines

What you should already know (Modern Code)

Java Fundamentals

- Variables, types (**int**, **double**, **boolean**)
- Control structures (**if**, **for**, **while**, **switch**)
- Methods (parameters, return types, scope)
- Arrays (1D and 2D)

Skills

- Strings and file I/O
- **Scanner** input, validation
- Recursion vs. iteration
- Debugging with IntelliJ
- Basic AI-assisted coding

If anything is unclear → review your Modern Code materials!